

```

import { useState, useEffect } from "react";

const FOODS_DB = [
  // — Grains & Pasta —
  { name: "White Rice", carbs: 45, sugar: 0, gi: 72, harmful: true, category: "Grains" },
  { name: "Brown Rice", carbs: 38, sugar: 0, gi: 50, harmful: false, category: "Grains" },
  { name: "White Pasta", carbs: 43, sugar: 1, gi: 50, harmful: true, category: "Pasta" },
  { name: "Whole Wheat Pasta", carbs: 35, sugar: 1, gi: 37, harmful: false, category: "Pasta" },
  { name: "Wheat (whole grain, cooked)", carbs: 30, sugar: 0, gi: 41, harmful: false, category: "Grains" },
  { name: "Semolina (cooked)", carbs: 40, sugar: 0, gi: 66, harmful: true, category: "Grains" },
  { name: "Oats / Oatmeal (rolled)", carbs: 27, sugar: 1, gi: 55, harmful: false, category: "Grains" },
  { name: "Oats (instant)", carbs: 30, sugar: 1, gi: 79, harmful: true, category: "Grains" },

  // — Bread & Cereals —
  { name: "White Bread", carbs: 28, sugar: 3, gi: 75, harmful: true, category: "Bread" },
  { name: "Whole Wheat Bread", carbs: 20, sugar: 2, gi: 51, harmful: false, category: "Bread" },
  { name: "Sourdough Bread", carbs: 22, sugar: 1, gi: 54, harmful: false, category: "Bread" },
  { name: "Corn Flakes", carbs: 36, sugar: 4, gi: 81, harmful: true, category: "Cereals" },
  { name: "Muesli (unsweetened)", carbs: 28, sugar: 6, gi: 40, harmful: false, category: "Cereals" },
  { name: "Sugary Cereal (e.g. Frosties)", carbs: 42, sugar: 28, gi: 77, harmful: true, category: "Cereals" },
  { name: "Bran Flakes", carbs: 30, sugar: 5, gi: 54, harmful: false, category: "Cereals" },
  { name: "Cornmeal Porridge (Ogi/Akamu)", carbs: 28, sugar: 1, gi: 68, harmful: true, category: "Cereals" },

  // — West African Staples —
  { name: "Yam (boiled)", carbs: 28, sugar: 0, gi: 54, harmful: false, category: "Tubers" },
  { name: "Yam (fried)", carbs: 35, sugar: 0, gi: 68, harmful: true, category: "Tubers" },
  { name: "Pounded Yam (pounded yam)", carbs: 44, sugar: 0, gi: 74, harmful: true, category: "Tubers" },
  { name: "Yam Flour (amala)", carbs: 42, sugar: 0, gi: 71, harmful: true, category: "Tubers" },
  { name: "Plantain Flour (amala abula)", carbs: 40, sugar: 2, gi: 65, harmful: true, category: "Tubers" },
  { name: "Boiled Plantain (unripe)", carbs: 30, sugar: 3, gi: 45, harmful: false, category: "Tubers" },
  { name: "Boiled Plantain (ripe)", carbs: 38, sugar: 14, gi: 60, harmful: true, category: "Tubers" },
  { name: "Fried Plantain - Dodo (ripe)", carbs: 40, sugar: 12, gi: 65, harmful: true, category: "Tubers" },
  { name: "Cassava Flakes (Garri)", carbs: 82, sugar: 0, gi: 84, harmful: true, category: "Tubers" },
  { name: "Eba (garri + hot water)", carbs: 75, sugar: 0, gi: 80, harmful: true, category: "Tubers" },
  { name: "Fufu (cassava-based)", carbs: 55, sugar: 0, gi: 72, harmful: true, category: "Tubers" },
  { name: "Tuwo Shinkafa (rice flour swallow)", carbs: 50, sugar: 0, gi: 73, harmful: true, category: "Tubers" },
  { name: "Akpu / Fufu (fermented cassava)", carbs: 48, sugar: 0, gi: 70, harmful: true, category: "Tubers" },

  // — Beans & Legumes —
  { name: "Black-eyed Beans (cooked)", carbs: 21, sugar: 3, gi: 33, harmful: false, category: "Beans" },
  { name: "Kidney Beans (cooked)", carbs: 22, sugar: 0, gi: 29, harmful: false, category: "Beans" },
  { name: "Lentils (cooked)", carbs: 20, sugar: 2, gi: 32, harmful: false, category: "Beans" },
  { name: "Chickpeas (cooked)", carbs: 27, sugar: 5, gi: 28, harmful: false, category: "Beans" },
  { name: "Brown Beans / Honey Beans", carbs: 20, sugar: 2, gi: 30, harmful: false, category: "Beans" }
];

```

```
{ name: "Baked Beans (tinned)", carbs: 25, sugar: 5, gi: 48, harmful: false, ca
{ name: "Moi Moi (steamed beans pudding)", carbs: 19, sugar: 2, gi: 32, harmful
{ name: "Akara (fried bean cakes)", carbs: 22, sugar: 1, gi: 45, harmful: false
{ name: "Pigeon Peas (cooked)", carbs: 23, sugar: 3, gi: 22, harmful: false, ca
{ name: "Soybeans (cooked)", carbs: 9, sugar: 0, gi: 18, harmful: false, catego
```

```
// — Eggs —
```

```
{ name: "Boiled Eggs", carbs: 1, sugar: 0, gi: 0, harmful: false, category: "Eg
{ name: "Fried Eggs (in little oil)", carbs: 1, sugar: 0, gi: 0, harmful: false
{ name: "Fried Eggs (deep-fried / excessive oil)", carbs: 1, sugar: 0, gi: 0, h
{ name: "Scrambled Eggs", carbs: 1, sugar: 0, gi: 0, harmful: false, category:
{ name: "Egg Omelette (plain)", carbs: 2, sugar: 0, gi: 0, harmful: false, cate
```

```
// — Meat & Protein —
```

```
{ name: "Chicken (grilled / skinless)", carbs: 0, sugar: 0, gi: 0, harmful: fal
{ name: "Chicken (fried with skin)", carbs: 8, sugar: 0, gi: 0, harmful: true,
{ name: "Red Meat — Beef (grilled/boiled)", carbs: 0, sugar: 0, gi: 0, harmful:
{ name: "Red Meat — Beef (processed/stew)", carbs: 5, sugar: 2, gi: 0, harmful:
{ name: "Red Meat — Goat (cooked)", carbs: 0, sugar: 0, gi: 0, harmful: false,
{ name: "Red Meat — Lamb (grilled)", carbs: 0, sugar: 0, gi: 0, harmful: false,
{ name: "Red Meat — Pork (lean, grilled)", carbs: 0, sugar: 0, gi: 0, harmful:
{ name: "Sausages (pork / beef)", carbs: 6, sugar: 2, gi: 28, harmful: true, ca
{ name: "Sausages (chicken / turkey, lean)", carbs: 4, sugar: 1, gi: 20, harmfu
{ name: "Sardines (in water)", carbs: 0, sugar: 0, gi: 0, harmful: false, categ
{ name: "Sardines (in tomato sauce)", carbs: 4, sugar: 3, gi: 0, harmful: false
{ name: "Sardines (in oil)", carbs: 0, sugar: 0, gi: 0, harmful: false, categor
{ name: "Fish — Grilled / Boiled", carbs: 0, sugar: 0, gi: 0, harmful: false, c
{ name: "Fish — Fried (battered)", carbs: 7, sugar: 0, gi: 0, harmful: true, ca
{ name: "Fish — Smoked / Dried (stockfish)", carbs: 0, sugar: 0, gi: 0, harmful
{ name: "Mackerel (grilled)", carbs: 0, sugar: 0, gi: 0, harmful: false, catego
{ name: "Salmon (grilled/baked)", carbs: 0, sugar: 0, gi: 0, harmful: false, ca
{ name: "Tuna (in water, tinned)", carbs: 0, sugar: 0, gi: 0, harmful: false, c
{ name: "Turkey (roasted, skinless)", carbs: 0, sugar: 0, gi: 0, harmful: false
```

```
// — Drinks —
```

```
{ name: "Sugar (white, 1 tsp)", carbs: 4, sugar: 4, gi: 65, harmful: true, cate
{ name: "Sugar (brown, 1 tsp)", carbs: 4, sugar: 4, gi: 64, harmful: true, cate
{ name: "Soda / Fizzy Drink (355ml)", carbs: 39, sugar: 39, gi: 65, harmful: tr
{ name: "Drink from Concentrate (Ribena, Squash)", carbs: 28, sugar: 27, gi: 60
{ name: "100% Fruit Juice (not from concentrate)", carbs: 26, sugar: 23, gi: 55
{ name: "Orange Juice (fresh, 250ml)", carbs: 26, sugar: 21, gi: 50, harmful: t
{ name: "Alcoholic Energy Drink (e.g. Four Loko)", carbs: 60, sugar: 55, gi: 70
{ name: "Non-Alcoholic Energy Drink (e.g. Red Bull)", carbs: 28, sugar: 27, gi:
{ name: "Beer (330ml)", carbs: 13, sugar: 0, gi: 66, harmful: true, category: "
{ name: "Wine (dry, 150ml)", carbs: 4, sugar: 1, gi: 0, harmful: false, categor
{ name: "Water", carbs: 0, sugar: 0, gi: 0, harmful: false, category: "Drinks &
{ name: "Unsweetened Tea", carbs: 0, sugar: 0, gi: 0, harmful: false, category:
```

```

{ name: "Unsweetened Black Coffee", carbs: 0, sugar: 0, gi: 0, harmful: false,
{ name: "Zobo (hibiscus drink, unsweetened)", carbs: 3, sugar: 0, gi: 15, harmf
{ name: "Zobo (sweetened)", carbs: 22, sugar: 20, gi: 55, harmful: true, catego

// — Snacks & Treats —
{ name: "Ice Cream (regular)", carbs: 24, sugar: 21, gi: 61, harmful: true, cat
{ name: "Ice Cream (low-sugar / diabetic)", carbs: 12, sugar: 5, gi: 30, harmfu
{ name: "Candy / Sweets", carbs: 44, sugar: 30, gi: 70, harmful: true, category
{ name: "Dark Chocolate (85%+)", carbs: 13, sugar: 4, gi: 23, harmful: false, c
{ name: "Milk Chocolate", carbs: 35, sugar: 30, gi: 49, harmful: true, category
{ name: "Chin Chin", carbs: 55, sugar: 10, gi: 70, harmful: true, category: "Sn
{ name: "Puff Puff", carbs: 40, sugar: 8, gi: 65, harmful: true, category: "Sna
{ name: "Peanuts (unsalted)", carbs: 7, sugar: 2, gi: 14, harmful: false, categ
{ name: "Popcorn (plain, air-popped)", carbs: 12, sugar: 0, gi: 55, harmful: fa
{ name: "French Fries", carbs: 35, sugar: 0, gi: 76, harmful: true, category: "
{ name: "Honey (1 tsp)", carbs: 17, sugar: 17, gi: 55, harmful: true, category:

// — Vegetables & Fruit —
{ name: "Sweet Potato (boiled)", carbs: 21, sugar: 7, gi: 44, harmful: false, c
{ name: "Watermelon", carbs: 12, sugar: 10, gi: 76, harmful: true, category: "V
{ name: "Banana (ripe)", carbs: 27, sugar: 14, gi: 51, harmful: false, category
{ name: "Apple", carbs: 25, sugar: 19, gi: 36, harmful: false, category: "Veg &
{ name: "Avocado", carbs: 9, sugar: 1, gi: 15, harmful: false, category: "Veg &
{ name: "Broccoli", carbs: 7, sugar: 2, gi: 15, harmful: false, category: "Veg
{ name: "Spinach", carbs: 4, sugar: 0, gi: 15, harmful: false, category: "Veg &
{ name: "Ugwu / Pumpkin Leaf", carbs: 3, sugar: 0, gi: 12, harmful: false, cate
{ name: "Bitter Leaf (Onugbu)", carbs: 2, sugar: 0, gi: 10, harmful: false, cat
{ name: "Tomatoes (fresh)", carbs: 4, sugar: 3, gi: 15, harmful: false, categor
{ name: "Cucumber", carbs: 4, sugar: 2, gi: 15, harmful: false, category: "Veg
{ name: "Pawpaw / Papaya", carbs: 11, sugar: 8, gi: 60, harmful: false, categor
{ name: "Mango (ripe)", carbs: 25, sugar: 22, gi: 51, harmful: true, category:
{ name: "Orange (whole fruit)", carbs: 15, sugar: 12, gi: 43, harmful: false, c
{ name: "Pineapple", carbs: 13, sugar: 10, gi: 59, harmful: false, category: "V

// — Dairy —
{ name: "Full-fat Yogurt (plain)", carbs: 8, sugar: 7, gi: 36, harmful: false,
{ name: "Flavoured Yogurt", carbs: 30, sugar: 24, gi: 52, harmful: true, catego
{ name: "Whole Milk (250ml)", carbs: 12, sugar: 12, gi: 31, harmful: false, cat
{ name: "Skimmed Milk (250ml)", carbs: 12, sugar: 12, gi: 32, harmful: false, c
{ name: "Almond Milk (unsweetened)", carbs: 2, sugar: 0, gi: 25, harmful: false
{ name: "Condensed Milk (1 tbs)", carbs: 21, sugar: 21, gi: 61, harmful: true,
];

const DAILY_LIMITS = { carbs: 130, sugar: 25 };
const MEAL_TIMES = ["Breakfast", "Morning Snack", "Lunch", "Afternoon Snack", "Di
const TABS = ["dashboard", "add food", "my log", "alc", "report", "reminders"];
const TAB_ICONS = { dashboard: "📊", "add food": "✚", "my log": "📅", alc: "🔴"

```

```

const inputStyle = {
  width: "100%", padding: "10px 13px", borderRadius: 10, border: "1px solid #1e3a
  background: "#0d1f35", color: "#e2e8f0", fontSize: 13, fontFamily: "inherit",
  outline: "none", boxSizing: "border-box",
};

const card = { background: "#0d1f35", borderRadius: 14, padding: "16px", border:

const GIBadge = ({ gi }) => {
  if (!gi) return <span style={{ fontSize: 10, color: "#334155" }}>No GI</span>;
  const color = gi >= 70 ? "#ef4444" : gi >= 56 ? "#f97316" : "#22c55e";
  return <span style={{ fontSize: 10, fontWeight: 700, color, background: color +
};

const Ring = ({ pct, color, label, val, limit, unit }) => {
  const r = 36, circ = 2 * Math.PI * r, dash = (Math.min(pct, 100) / 100) * circ;
  return (
    <div style={{ display: "flex", flexDirection: "column", alignItems: "center",
    <div style={{ position: "relative", width: 90, height: 90 }}>
      <svg width={90} height={90} style={{ transform: "rotate(-90deg)" }}>
        <circle cx={45} cy={45} r={r} fill="none" stroke="#1e293b" strokeWidth=
        <circle cx={45} cy={45} r={r} fill="none" stroke={color} strokeWidth={7
          strokeDasharray={` ${dash} ${circ}`} strokeLinecap="round"
          style={{ transition: "stroke-dasharray 0.5s ease" }} />
      </svg>
      <div style={{ position: "absolute", inset: 0, display: "flex", flexDirect
        <span style={{ fontSize: 19, fontWeight: 800, color, fontFamily: "monos
        <span style={{ fontSize: 9, color: "#64748b" }}>{unit}</span>
      </div>
    </div>
    <div style={{ textAlign: "center" }}>
      <div style={{ fontSize: 11, fontWeight: 700, color: "#cbd5e1" }}>{label}<
      <div style={{ fontSize: 10, color: "#475569" }}>of {limit}{unit} {pct} >=
    </div>
  </div>
);
};

export default function GlycoGuard() {
  const [log, setLog] = useState([]);
  const [customFoods, setCustomFoods] = useState([]);
  const [search, setSearch] = useState("");
  const [tab, setTab] = useState("dashboard");
  const [addedIds, setAddedIds] = useState(new Set());
  const [notif, setNotif] = useState(null);
  const [showHarmful, setShowHarmful] = useState(false);
  const [selectedMeal, setSelectedMeal] = useState("Breakfast");

```

```

const [reminders, setReminders] = useState([
  { id: 1, label: "Breakfast log", time: "08:00", active: true },
  { id: 2, label: "Lunch log", time: "13:00", active: true },
  { id: 3, label: "Dinner log", time: "19:00", active: false },
  { id: 4, label: "Blood sugar check", time: "21:00", active: false },
]);

const [bloodSugar, setBloodSugar] = useState("");
const [alcReadings, setAlcReadings] = useState([]);
const [patientName, setPatientName] = useState("Patient");
const [patientAge, setPatientAge] = useState("");
const [diabetesType, setDiabetesType] = useState("Type 2");
const [gpNotes, setGpNotes] = useState("");
const [newRemLabel, setNewRemLabel] = useState("");
const [newRemTime, setNewRemTime] = useState("09:00");
const [cf, setCf] = useState({ name: "", carbs: "", sugar: "", gi: "", harmful:
const [cfError, setCfError] = useState("");
const [selectedCategory, setSelectedCategory] = useState("All");

const CATEGORIES = ["All", ...Array.from(new Set(FOODS_DB.map(f => f.category))
const allFoods = [...FOODS_DB, ...customFoods];
const filtered = allFoods.filter(f =>
  f.name.toLowerCase().includes(search.toLowerCase()) &&
  (selectedCategory === "All" || f.category === selectedCategory)
);
const totalCarbs = log.reduce((s, f) => s + Number(f.carbs), 0);
const totalSugar = log.reduce((s, f) => s + Number(f.sugar), 0);
const harmfulCount = log.filter(f => f.harmful).length;
const byMeal = MEAL_TIMES.reduce((acc, m) => { acc[m] = log.filter(f => f.meal

const notify = (type, msg, dur = 2500) => {
  setNotif({ type, msg });
  setTimeout(() => setNotif(null), dur);
};

const addFood = (food) => {
  const entry = { ...food, id: Date.now(), meal: selectedMeal, time: new Date()
  setLog(prev => [...prev, entry]);
  setAddedIds(prev => new Set([...prev, food.name]));
  if (food.harmful) notify("warn", `⚠️ "${food.name}" is harmful for diabetics.
  else notify("ok", `✓ "${food.name}" added to ${selectedMeal}.`);
};

const removeFood = (id) => {
  setLog(prev => {
    const removed = prev.find(f => f.id === id);
    const next = prev.filter(f => f.id !== id);
    if (removed && !next.some(f => f.name === removed.name))

```

```

        setAddedIds(p => { const n = new Set(p); n.delete(removed.name); return n
        return next;
    });
};

const logBloodSugar = () => {
    const val = parseFloat(bloodSugar);
    if (!val || val < 2 || val > 35) return notify("warn", "Enter a valid blood s
    const mgdl = val * 18;
    const estimatedA1C = ((mgdl + 46.7) / 28.7).toFixed(1);
    setA1cReadings(prev => [...prev, { id: Date.now(), val, mgdl: Math.round(mgdl
    setBloodSugar("");
    notify("ok", `Logged ${val} mmol/L → Est. A1C: ${estimatedA1C}%`);
};

const avgA1C = a1cReadings.length ? (a1cReadings.reduce((s, r) => s + parseFloa
const a1cStatus = !avgA1C ? null : avgA1C < 5.7 ? { label: "Normal", color: "#2

const generateReport = () => {
    const date = new Date().toLocaleDateString("en-GB", { day: "numeric", month:
    return [
        `GLYCOGUARD - PATIENT DIETARY REPORT`,
        `Generated: ${date}`,
        `${"-".repeat(44)}`,
        `Patient: ${patientName} | Age: ${patientAge || "N/A"} | Diabetes: ${di
        `,
        `DAILY NUTRITIONAL SUMMARY`,
        `  Total Carbohydrates : ${totalCarbs}g (Daily limit: ${DAILY_LIMITS.carbs
        `  Total Sugar          : ${totalSugar}g (Daily limit: ${DAILY_LIMITS.sugar
        `  Foods Logged         : ${log.length}`,
        `  Harmful Items         : ${harmfulCount}`,
        `,
        `BLOOD SUGAR & ESTIMATED A1C`,
        a1cReadings.length ? `  Readings: ${a1cReadings.length} | Avg A1C: ${avgA
        `,
        `MEAL BREAKDOWN`,
        ...MEAL_TIMES.flatMap(m => {
            const items = byMeal[m];
            if (!items.length) return [];
            return [
                `${m}:`,
                ...items.map(f => `    • ${f.name} - Carbs: ${f.carbs}
            ],
        ],
        `HARMFUL FOODS CONSUMED`,
        ...(log.filter(f => f.harmful).length ? log.filter(f => f.harmful).map(f =>
        `,
        `GP NOTES`,
        gpNotes || "  No additional notes.",

```

```

    ``,
    `-----`,
    `This report was generated by GlycoGuard.`,
    `Not a substitute for professional medical advice.`,
  ].join("\n");
};

const downloadReport = () => {
  const blob = new Blob([generateReport()], { type: "text/plain" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url; a.download = `GlycoGuard_Report_${new Date().toISOString().slice(0, 10)}.pdf`;
  a.click(); URL.revokeObjectURL(url);
  notify("ok", "📄 Report downloaded!");
};

const addCustomFood = () => {
  if (!cf.name.trim()) return setCfError("Food name is required.");
  if (cf.carbs === "" || cf.sugar === "") return setCfError("Carbs and sugar are required.");
  if (cf.harmful && !cf.reason.trim()) return setCfError("Please explain why this food is harmful.");
  setCustomFoods(prev => [...prev, { name: cf.name.trim(), carbs: Number(cf.carbs), sugar: Number(cf.sugar), gi: Number(cf.gi), harmful: cf.harmful, reason: cf.reason }]);
  setCfError("");
  notify("ok", `✓ "${cf.name.trim()}" added to food library!`);
};

return (
  <div style={{ minHeight: "100vh", background: "#060e1a", fontFamily: "'DM Sans', sans-serif" }}>
    <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;500;600;700;800;900&display=block" />

    {/* Toast */}
    {notif && (
      <div style={{
        position: "fixed", top: 14, left: "50%", transform: "translateX(-50%)",
        background: notif.type === "warn" ? "#450a0a" : "#052e16",
        border: `1px solid ${notif.type === "warn" ? "#ef4444" : "#22c55e"}`,
        color: notif.type === "warn" ? "#fca5a5" : "#86efac",
        padding: "9px 18px", borderRadius: 10, fontSize: 12, fontWeight: 600,
        boxShadow: "0 8px 32px #000b", whiteSpace: "nowrap", maxWidth: "90vw"
      }}>{notif.msg}</div>
    )}

    <div style={{ maxWidth: 560, margin: "0 auto", padding: "0 16px" }}>
      {/* Header */}
      <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center" }}>
        <div>
          <div style={{ fontSize: 23, fontWeight: 800, color: "#f1f5f9", letterSpacing: 0.5, margin-bottom: 10px }}>
            GlycoGuard
          
```

```

        Glyco<span style={{ color: "#34d399" }}>Guard</span>
        <span style={{ fontSize: 10, fontWeight: 400, color: "#334155", mar
    </div>
    <div style={{ fontSize: 10, color: "#1e3a5f", marginTop: 1 }}>Diabeti
    </div>
    <div style={{ background: harmfulCount > 0 ? "#450a0a" : "#052e16", bor
        <div style={{ fontSize: 19, fontWeight: 800, color: harmfulCount > 0
            <div style={{ fontSize: 9, color: harmfulCount > 0 ? "#fca5a5" : "#4a
            </div>
        </div>
    </div>

    { /* Tabs */ }
    <div style={{ display: "flex", gap: 5, marginTop: 16, overflowX: "auto",
        { TABS.map(t => (
            <button key={t} onClick={() => setTab(t)} style={{
                flexShrink: 0, padding: "6px 9px", borderRadius: 9, border: "none",
                background: tab === t ? "#1e3a5f" : "#0d1f35",
                color: tab === t ? "#7dd3fc" : "#334155",
                fontWeight: tab === t ? 700 : 500, fontSize: 10, fontFamily: "inher
                display: "flex", flexDirection: "column", alignItems: "center", gap
            }}>
                <span style={{ fontSize: 15 }}>{TAB_ICONS[t]}</span>
                <span style={{ textTransform: "capitalize" }}>{t}</span>
            </button>
        ))}
    </div>

    { /* == DASHBOARD == */ }
    { tab === "dashboard" && (
        <div style={{ marginTop: 16, display: "flex", flexDirection: "column",
            <div style={{ ...card, display: "flex", justifyContent: "space-around
                <Ring pct={({totalCarbs / DAILY_LIMITS.carbs) * 100} color="#38bdf8"
                <div style={{ width: 1, height: 80, background: "#1e3a5f" }} />
                <Ring pct={({totalSugar / DAILY_LIMITS.sugar) * 100} color="#f472b6"
            </div>

        <div style={{ ...card, background: harmfulCount > 0 ? "#1c0606" : "#0
            <span style={{ fontSize: 26 }}>{harmfulCount > 0 ? "⚠️" : "✅"}</span>
            <div>
                <div style={{ fontWeight: 700, fontSize: 13, color: harmfulCount
                    {harmfulCount > 0 ? `${harmfulCount} harmful item${harmfulCount
                </div>
                <div style={{ fontSize: 11, color: "#475569", marginTop: 2 }}>
                    {harmfulCount > 0 ? "Go to My Log → remove harmful items." : "G
                </div>
            </div>
        </div>
    </div>

```



```

{avgA1C && (
  <div style={{ ...card, display: "flex", alignItems: "center", justi
    <div>
      <div style={{ fontSize: 10, color: "#475569" }}>EST. AVERAGE A1
      <div style={{ fontSize: 38, fontWeight: 800, color: alcStatus.c
      <div style={{ fontSize: 11, fontWeight: 700, color: alcStatus.c
    </div>
    <div style={{ display: "flex", flexDirection: "column", gap: 4 }}
      {[["< 5.7%", "Normal", "#22c55e"], ["5.7-6.4%", "Pre-diabetic",
        <div key={l} style={{ fontSize: 10, color: c, background: c +
          )}}
      </div>
    </div>
  </div>
)}

<div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 8
  {[["🍌", log.length, "Foods Logged"], ["✅", log.filter(f => !f.ha
    <div key={label} style={card}>
      <div style={{ fontSize: 18 }}>{icon}</div>
      <div style={{ fontSize: 22, fontWeight: 800, color: "#f1f5f9",
      <div style={{ fontSize: 10, color: "#475569", marginTop: 1 }}>{
    </div>
  )}}
</div>

<div style={card}>
  <div style={{ fontSize: 11, fontWeight: 700, color: "#7dd3fc", marg
    {MEAL_TIMES.map(m => {
      const items = byMeal[m];
      if (!items.length) return null;
      const mCarbs = items.reduce((s, f) => s + Number(f.carbs), 0);
      return (
        <div key={m} style={{ display: "flex", justifyContent: "space-b
          <span style={{ fontSize: 12, fontWeight: 600, color: "#e2e8f0
          <span style={{ fontSize: 11, fontFamily: "monospace", color:
        </div>
      );
    )}}
    {log.length === 0 && <div style={{ fontSize: 11, color: "#1e3a5f",
  </div>
</div>
)}

{/* === ADD FOOD === */}
{tab === "add food" && (
  <div style={{ marginTop: 16, display: "flex", flexDirection: "column",

```

```

<div>
  <div style={{ fontSize: 10, color: "#475569", marginBottom: 6, font
  <div style={{ display: "flex", flexWrap: "wrap", gap: 5 }}>
    {MEAL_TIMES.map(m => (
      <button key={m} onClick={() => setSelectedMeal(m)} style={{
        padding: "6px 11px", borderRadius: 8, border: `1px solid ${se
        background: selectedMeal === m ? "#0c2a45" : "transparent",
        color: selectedMeal === m ? "#7dd3fc" : "#475569",
        fontSize: 11, fontWeight: selectedMeal === m ? 700 : 500, cur
      }}>{m}</button>
    ))}
  </div>
</div>

```

```

<input type="text" placeholder="🔍 Search foods..." value={search} c

```

```

{/* Category filter */}
<div style={{ display: "flex", gap: 5, overflowX: "auto", paddingBott
  {CATEGORIES.map(cat => (
    <button key={cat} onClick={() => setSelectedCategory(cat)} style=
      flexShrink: 0, padding: "5px 10px", borderRadius: 8,
      border: `1px solid ${selectedCategory === cat ? "#a78bfa" : "#1
      background: selectedCategory === cat ? "#2dlb69" : "transparent
      color: selectedCategory === cat ? "#c4b5fd" : "#334155",
      fontSize: 10, fontWeight: selectedCategory === cat ? 700 : 500,
      cursor: "pointer", fontFamily: "inherit", whiteSpace: "nowrap"
    }}>{cat}</button>
  ))}
</div>

```

```

<div style={{ display: "flex", justifyContent: "space-between", align
  <span style={{ fontSize: 10, color: "#334155" }}>{filtered.filter(f
  <button onClick={() => setShowHarmful(p => !p)} style={{ padding: "
    {showHarmful ? "⚠️ Harmful only" : "All foods"}
  </button>
</div>

```

```

<div style={{ display: "flex", flexDirection: "column", gap: 7, maxHe
  {filtered.filter(f => !showHarmful || f.harmful).map((food, i) => (
    <div key={food.name + i} style={{ background: food.harmful ? "#1c
      <div style={{ flex: 1 }}>
        <div style={{ display: "flex", alignItems: "center", gap: 6,
          <span style={{ fontWeight: 700, fontSize: 13, color: "#f1f5
            {food.harmful && <span style={{ fontSize: 9, color: "#fca5a
              {food.custom && <span style={{ fontSize: 9, color: "#a78bfa
            </div>
          <div style={{ marginTop: 3, display: "flex", gap: 8, alignIte

```

```

        <span style={{ fontSize: 10, color: "#38bdf8", fontFamily:
        <span style={{ fontSize: 10, color: "#f472b6", fontFamily:
        <GIBadge gi={food.gi} />
      </div>
      {food.harmful && <div style={{ marginTop: 4, fontSize: 10, co
    </div>
    <button onClick={() => addFood(food)} style={{ background: adde
      {addedIds.has(food.name) ? "✓" : "+ Add"}
    </button>
  </div>
  )}}
</div>

{/* Custom Food */}
<div style={{ ...card, marginTop: 4, border: "1px solid #2d1b69" }}>
  <div style={{ fontSize: 11, fontWeight: 700, color: "#a78bfa", marg
  <div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
    <input placeholder="Food name *" value={cf.name} onChange={e => s
    <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr"
      <input placeholder="Carbs (g) *" type="number" value={cf.carbs}
      <input placeholder="Sugar (g) *" type="number" value={cf.sugar}
      <input placeholder="GI score" type="number" value={cf.gi} onCha
    </div>
    <label style={{ display: "flex", alignItems: "center", gap: 8, fo
      <input type="checkbox" checked={cf.harmful} onChange={e => setC
      Mark as harmful for diabetics
    </label>
    {cf.harmful && <input placeholder="Why is it harmful? *" value={c
    {cf.error && <div style={{ fontSize: 11, color: "#fca5a5" }}>△ {cf
    <button onClick={addCustomFood} style={{ background: "#4c1d95", b
  </div>
</div>
</div>
)}

{/* === MY LOG === */}
{tab === "my log" && (
  <div style={{ marginTop: 16, display: "flex", flexDirection: "column",
    {log.length === 0 ? (
      <div style={{ textAlign: "center", color: "#1e3a5f", padding: "60px
        <div style={{ fontSize: 36 }}>🕒</div>
        <div style={{ fontSize: 13, marginTop: 10, color: "#334155" }}>No
      </div>
    ) : (
      <>
      {harmfulCount > 0 && (
        <div style={{ background: "#1c0606", border: "1px solid #7f1d1d

```

[illegible]

```

        Enter your blood glucose level. GlycoGuard estimates your A1C usi
    </div>
    <div style={{ display: "flex", gap: 8 }}>
        <input type="number" step="0.1" min="2" max="35" placeholder="e.g
        <button onClick={logBloodSugar} style={{ background: "#831843", b
    </div>
</div>

{avgA1C && (
    <div style={{ ...card, border: `2px solid ${alcStatus.color}44` }}>
        <div style={{ display: "flex", justifyContent: "space-between", a
            <div>
                <div style={{ fontSize: 10, color: "#475569" }}>ESTIMATED AVE
                <div style={{ fontSize: 46, fontWeight: 800, color: alcStatus
                <div style={{ fontSize: 12, fontWeight: 700, color: alcStatus
                <div style={{ fontSize: 10, color: "#475569", marginTop: 2 }}
            </div>
            <div style={{ display: "flex", flexDirection: "column", gap: 5
                {[["< 5.7%", "Normal", "#22c55e"], ["5.7-6.4%", "Pre-diabetic
                <div key={1} style={{ fontSize: 10, color: c, background: c
                    )}}
            </div>
        </div>
    </div>
)}

{alcReadings.length > 0 && (
    <div style={card}>
        <div style={{ fontSize: 11, fontWeight: 700, color: "#7dd3fc", ma
        <div style={{ display: "flex", flexDirection: "column", gap: 5 }}
            {[...alcReadings].reverse().map(r => {
                const c = r.alc < 5.7 ? "#22c55e" : r.alc < 6.5 ? "#f97316" :
                return (
                    <div key={r.id} style={{ display: "flex", justifyContent: "
                        <div>
                            <span style={{ fontWeight: 700, color: "#e2e8f0", fontF
                            <span style={{ fontSize: 10, color: "#334155", marginLe
                        </div>
                        <span style={{ fontSize: 12, fontWeight: 700, color: c }}
                    </div>
                );
            })}
        </div>
    </div>
)}

{alcReadings.length === 0 && <div style={{ textAlign: "center", color

```

```

<div style={{ ...card, background: "#071a0e", border: "1px solid #145
  <div style={{ fontSize: 10, color: "#4ade80", fontWeight: 700, marg
    <div style={{ fontSize: 11, color: "#475569", lineHeight: 1.6 }}>
      eAG formula: <span style={{ fontFamily: "monospace", color: "#94a
        For clinical accuracy, ask your GP for an HbA1c blood test.
    </div>
  </div>
</div>
)}

{/* === GP REPORT === */}
{tab === "report" && (
  <div style={{ marginTop: 16, display: "flex", flexDirection: "column",
    <div style={card}>
      <div style={{ fontSize: 11, fontWeight: 700, color: "#34d399", marg
        <div style={{ display: "flex", flexDirection: "column", gap: 7 }}>
          <input placeholder="Patient name" value={patientName} onChange={e
            <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", ga
              <input placeholder="Age" type="number" value={patientAge} onCha
                <select value={diabetesType} onChange={e => setDiabetesType(e.t
                  <option>Type 1</option><option>Type 2</option><option>Gestati
                </select>
            </div>
          <textarea placeholder="Additional GP notes..." value={gpNotes} on
        </div>
      </div>

    <div style={{ ...card, background: "#060e1a" }}>
      <div style={{ fontSize: 11, fontWeight: 700, color: "#7dd3fc", marg
        <pre style={{ fontSize: 10, color: "#475569", fontFamily: "DM Mono,
      </div>

    <button onClick={downloadReport} style={{ background: "linear-gradien
      📄 Download GP Report (.txt)
    </button>
    <div style={{ fontSize: 10, color: "#1e3a5f", textAlign: "center" }}>
  </div>
)}

{/* === REMINDERS === */}
{tab === "reminders" && (
  <div style={{ marginTop: 16, display: "flex", flexDirection: "column",
    <div style={{ fontSize: 11, color: "#334155", lineHeight: 1.5 }}>
      Set meal and check-up reminders below. Visual reminders only – use
    </div>
  </div>
)}

```

```

<div style={{ ...card, border: "1px solid #78350f" }}>
  <div style={{ fontSize: 11, fontWeight: 700, color: "#fbbf24", marg
  <div style={{ display: "flex", flexDirection: "column", gap: 7 }}>
    <input placeholder="Label (e.g. Post-meal check)" value={newRemLa
    <div style={{ display: "flex", gap: 7 }}>
      <input type="time" value={newRemTime} onChange={e => setNewRemT
      <button onClick={() => { if (!newRemLabel.trim()) return; setRe
    </div>
  </div>
</div>

<div style={{ display: "flex", flexDirection: "column", gap: 7 }}>
  {reminders.map(r => (
    <div key={r.id} style={{ ...card, background: r.active ? "#0a1a2e
    <div style={{ display: "flex", alignItems: "center", gap: 10 }}
      <span style={{ fontSize: 18 }}>{r.active ? "🔔" : "🔕"}</span>
    <div>
      <div style={{ fontSize: 13, fontWeight: 700, color: r.activ
      <div style={{ fontSize: 12, color: "#38bdf8", fontFamily: "
    </div>
  </div>
  <div style={{ display: "flex", gap: 5 }}>
    <button onClick={() => setReminders(p => p.map(x => x.id ===
    <button onClick={() => setReminders(p => p.filter(x => x.id !
  </div>
  </div>
  )))
</div>

<div style={card}>
  <div style={{ fontSize: 11, fontWeight: 700, color: "#7dd3fc", marg
  <div style={{ display: "flex", flexWrap: "wrap", gap: 6 }}>
    {[["Morning check", "07:30"], ["Before breakfast", "08:00"], ["Po
    <button key={label} onClick={() => { setReminders(p => [...p, {
    )}}
  </div>
</div>
</div>
  )}
</div>
</div>
);
}

```